

Synthèse du chapitre 2 : Analyse Lexicale

Comprendre comment le compilateur reconnaît les **unités lexicales (tokens)** à partir du texte source brut, en s'appuyant sur des outils formels comme les **expressions régulières** et les **automates finis**.

Concepts fondamentaux

1. Définition

L'**analyse lexicale** est la première phase du compilateur. Elle transforme le **flux de caractères** du programme source en une **séquence de tokens** significatifs.

2. Lexème, Token et Atome lexical

- **Lexème** : suite concrète de caractères dans le code (ex. "while", "x123")
- **Token** : catégorie abstraite à laquelle le lexème appartient (ex. WHILE, ID)
- **Attribut** : informations supplémentaires (ex. valeur numérique, nom de variable)

3. Exemple

```
int x = 5 + 3;
```

Produit les tokens :

```
<INT_KW> <ID:x> <ASSIGN> <NUM:5> <PLUS> <NUM:3> <SEMI>
```

Fonctionnement du lexeur

Le **lexeur (ou scanner)** :

- Ignore les blancs, commentaires, sauts de ligne
- Retourne les tokens un par un à l'analyseur syntaxique
- Peut construire une **table des symboles** (pour les identifiants)
- Doit détecter et signaler les **erreurs lexicales**

Définition des tokens avec des expressions régulières

Chaque type de token est défini par une **expression régulière** (regex), par exemple :

Token	Expression régulière
ID	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>
NUM	<code>[0-9]+</code>
IF	<code>if</code>
WHILE	<code>while</code>

OP	[+* / -]
SEMI	;

Implémentation via automates finis

Les expressions régulières peuvent être transformées en **automates finis déterministes (DFA)** pour reconnaître efficacement les lexèmes.

Étapes typiques :

1. Définir des regex pour chaque token
2. Construire un **automate non déterministe (NFA)**
3. Le convertir en **DFA**
4. Utiliser le DFA pour lire les caractères en temps linéaire

Outils associés

Outil	Rôle
Lex / Flex	Génèrent automatiquement le code du scanner à partir des regex
Python	Modules comme <code>re</code> ou <code>PLY</code> (<code>t_ID</code> , <code>t_NUM</code>)
ANTLR / JFlex	Plus modernes et puissants pour des langages complexes

Rôle de la table des symboles

Le lexeur peut alimenter une **table des symboles** avec :

- les noms d'identifiants
- leur position dans le code
- leur type (à déterminer plus tard)
- leur portée (gérée par l'analyse sémantique)

Gestion des erreurs lexicales

- Caractères illégaux (ex. @, #, si non autorisés)
- Identifiants trop longs
- Constantes mal formées

La détection doit être **rapide, précise**, et permettre de **continuer l'analyse** quand possible.

Résumé visuel

